

AI 부트캠프 · 팀 프로젝트

알약 객체 탐지 모델 고도화 프로젝트

재현 가능한 실험으로 알약 탐지 성능을 끌어올리는 모델 개발 · 실험 관리 시스템

YOLO 기반 객체 탐지

실험 관리 GUI

Kaggle 비공개 대회

발표 순서

01 과제 정의

무엇을 예측하고 어떻게 채점되는가

03 데이터

어노테이션 구조, EDA, 전처리와 증강

05 실험 관리 시스템

설정 GUI와 결과 대시보드

02 기본 개념 정리

탐지, IoU, mAP, 전이학습, 임계값

04 모델 개발

베이스라인, 파인튜닝, 하이퍼파라미터

06 실행과 마무리

오류 분석, 제출 관리, 최종 데모

프로젝트 목표

재현 가능한 학습·평가·오류 분석을 통해
알약 객체 탐지 성능을 개선하는 **모델 개발 및 실험 관리 시스템**

목표로 삼는 것

어떤 기법이 성능을 올렸고, 왜 올랐으며, 어디에서 여전히 실패하는지 설명할 수 있는 상태

목표가 아닌 것

근거 없이 리더보드 점수 한 번을 높게 만드는 것, 그리고 일반 사용자용 복약 서비스를 만드는 것

Kaggle 과제 정의

이미지 한 장에서 최대 4개의 알약을 찾아, 각각의 **클래스**와 **바운딩 박스**를 함께 예측합니다.

입력

알약 이미지 1장

여러 알약이 함께 놓여 있거나, 겹치거나,
조명·배경이 다를 수 있습니다.



출력

최대 4개의 예측

각 예측 = 클래스 1개 + 박스 좌표 4개 +
신뢰도 점수 1개



채점

클래스 + 위치 동시 판정

종류만 맞거나 위치만 맞으면 정답으로 인
정되지 않습니다.

제출 형식과 채점 방식

예시 형식

submission.csv

image_id, class_id, conf, x_min, y_min, x_max, y_max

img_0001, 12, 0.94, 210, 138, 402, 331

img_0001, 07, 0.88, 455, 210, 640, 396

img_0002, 12, 0.91, 118, 96, 318, 288

...

mAP@0.5

박스가 정답과 절반 이상 겹치면 정답으로 인정하는 기준입니다.

mAP@0.5:0.95

겹침 기준을 0.5부터 0.95까지 올려 가며 평균 낸, 훨씬 엄격한 기준입니다.

프로젝트 범위

범위 안

알약 탐지 모델의 학습과 성능 개선

실험 설정을 만들고 기록하는 웹 GUI

학습 결과를 비교하는 대시보드

오류 분석과 Kaggle 제출 관리

범위 밖

일반 사용자용 복약 관리 앱

약품 추천 또는 복용 지도 기능

완성형 MLOps 플랫폼 구축

의료적 판단에 사용 가능하다는 주장

SECTION 02

기본 개념 정리

이후 모든 실험 논의가 이 여덟 가지 구분 위에서 이루어집니다.

01 이미지 분류와 객체 탐지

분류는 “무엇인가”만 답하고, 탐지는 “무엇이 어디에 몇 개 있는가”까지 답합니다.

이미지 분류

클래스 12

알약 이미지

출력은 라벨 하나. 알약이 두 개 있어도 하나로만 답합니다.

객체 탐지 — 우리 과제

12 · 0.94

07 · 0.88

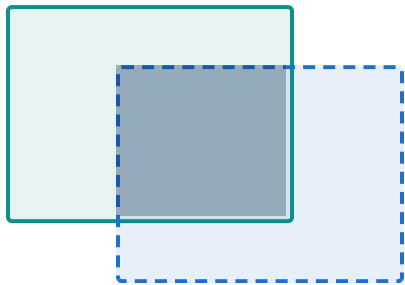
알약 이미지

출력은 박스 목록. 개수, 위치, 클래스, 신뢰도를 모두 답합니다.

02 바운딩 박스와 IoU

IoU는 예측 박스와 정답 박스가 겹친 넓이 ÷ 합친 넓이입니다. 이 값이 기준을 넘어야 정답으로 셉니다.

정답 박스



예측 박스

0.28

거의 빗나감 — 오탐으로 처리됩니다

0.62

@0.5 기준에서는 정답, @0.75 기준에서는 오답

0.91

거의 완벽한 위치 — 엄격한 기준에서도 정답

03 Precision과 Recall

정밀도는 “찾았다고 한 것이 맞았는가”, 재현율은 “있는 것을 다 찾았는가”를 잹니다.

Precision · 정밀도

모델이 알약이라고 표시한 박스 중 실제로 맞은 비율

낮으면 → 없는 알약을 자주 찾아냅니다 (오탐 과다)

Recall · 재현율

실제로 있는 알약 중 모델이 찾아낸 비율

낮으면 → 있는 알약을 놓칩니다 (미탐 과다)

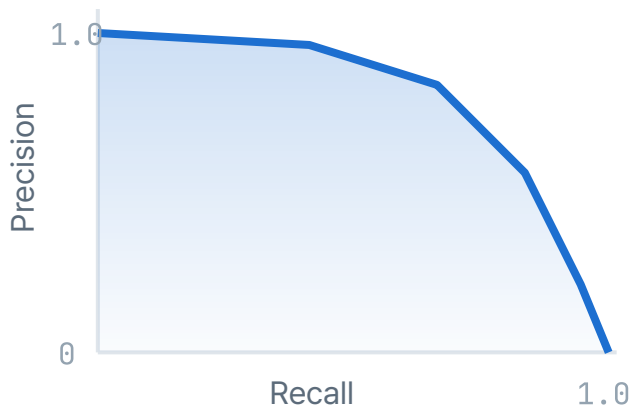
트레이드오프

신뢰도 임계값을 올리면 정밀도는 오르고 재현율은 내려갑니다. 한쪽만 좋아진 결과는 개선이 아니라 **기준선 이동**일 수 있습니다.

04 mAP@0.5와 mAP@0.5:0.95

정밀도-재현율 곡선의 아래 면적을 클래스별로 구해 평균 낸 값이 mAP입니다.

Precision - Recall 곡선



mAP@0.5

IoU 0.5 하나의 기준. 클래스를 제대로 구분하는지 보기에 좋습니다.

mAP@0.5:0.95

0.5부터 0.95까지 10개 기준의 평균. 박스 정확도까지 반영합니다.

두 값의 격차가 크면 — 클래스는 맞지만 박스가 헛겁다는 신호입니다.

05 스크래치 학습 · 전이학습 · 파인튜닝

방식 A

스크래치 학습

빈 모델에서 시작합니다. 데이터와 학습 시간이 크게 필요하고, 부트캠프 기간에는 현실적이지 않습니다.

비교용으로만 1회 시도

방식 B

사전학습 가중치만 사용

공개 데이터로 학습된 모델을 그대로 씁니다. 일반 사물은 잘 찾지만 우리 알약 클래스는 모릅니다.

출발점일 뿐, 답이 아님

방식 C

파인튜닝

사전학습 가중치에서 출발해 우리 알약 데이터로 이어서 학습합니다. 적은 데이터로도 빠르게 수렴합니다.

기본 전략으로 채택

전이학습은 “남이 배운 것을 가져온다”는 **개념**이고, 파인튜닝은 그것을 우리 데이터로 이어서 학습하는 **방법**입니다.

06 학습 파라미터와 추론 임계값

앞은 모델의 실력을 바꾸고, 뒤는 이미 학습된 모델의 답을 어디서 자를지 정합니다.

학습 파라미터

learning rate

batch size

epochs

image size

optimizer

augmentation

바꾸면 반드시 재학습이 필요합니다. 새로운 실험으로 기록합니다.

추론 임계값

confidence threshold

NMS IoU

max detections

재학습 없이 즉시 바꿀 수 있습니다. 같은 모델의 출력 후처리일 뿐입니다.

해석 주의 임계값만 낮춰서 오른 mAP는 모델 개선이 아닙니다. 실험 기록에 두 종류를 반드시 분리해 남깁니다.

07 데이터 증강과 TTA

학습 시점

데이터 증강 · Augmentation

학습 이미지를 뒤집고, 회전하고, 밝기와 대비를 바꿔 가며 넣습니다.

목적 — 다양한 촬영 조건에 견디는 모델을 만듭니다. 학습 시간이 늘어납니다.

추론 시점

TTA · Test-Time Augmentation

예측할 때 같은 이미지를 여러 변형으로 넣고 결과를 합칩니다.

목적 — 예측을 안정시킵니다. 모델은 그대로이고 추론 시간이 늘어납니다.

증강은 **모델의 실력**을 바꾸고, TTA는 **한 번의 답**을 다듬습니다. 실험 기록에서 두 효과를 섞어 보고하지 않습니다.

08 로컬 검증 점수와 리더보드 점수

로컬 검증

우리가 떼어 둔 검증 데이터로 잭니다. 횟수 제한이 없고, 틀린 이미지를 직접 열어 볼 수 있습니다.

Kaggle 리더보드

정답을 볼 수 없는 데이터로 잭니다. 제출 횟수가 제한되고, 어디서 틀렸는지 알 수 없습니다.

둘 다 상승

신뢰할 수 있는 개선입니다. 채택합니다.

로컬만 상승

검증 분할이 실제 분포와 다를 수 있습니다.

리더보드만 상승

우연이거나 리더보드 과적합을 의심합니다.

SECTION 03

데이터

모델을 바꾸기 전에 데이터를 먼저 이해합니다.

데이터셋과 어노테이션 구조

예시 형식

이미지

알약이 1~4개 놓인 사진. 배경·조명·촬영 각도가 제각각입니다.

어노테이션

이미지별 박스 목록. 클래스 ID와 좌표 4개가 한 쌍입니다.

클래스 매핑

클래스 ID와 실제 알약 이름을 잇는 표. 제출과 시각화에 모두 씁니다.

YOLO 형식 - 이미지당 .txt 한 개

```
class x_center y_center width height
12 0.418 0.362 0.196 0.212
07 0.741 0.588 0.174 0.188
```

좌표는 0-1로 정규화

COCO 형식 - 전체 JSON 한 개

```
bbox = [x_min, y_min, width, height]
절대 픽셀 좌표
```

두 형식은 좌표 기준이 다릅니다. 어떤 형식이 오든 **학습 전 단일 형식으로 변환**하는 단계를 파이프라인에 고정합니다.

EDA 전략

확인 항목마다 “그래서 무엇을 바꿀 것인가”를 미리 정해 둡니다.

확인 항목	결과에 따른 결정
클래스별 이미지 수	치우침이 크면 계층 분할과 클래스 가중치를 검토합니다
이미지당 알약 개수	다수 객체가 많으면 NMS 설정과 최대 탐지 수를 조정합니다
박스 크기 분포	작은 객체가 많으면 입력 해상도를 올립니다
박스 겹침 정도	겹침이 잦으면 NMS IoU 임계값을 실험 변수로 넣습니다
밝기 · 배경 · 반사	편차가 크면 색상·밝기 증강 범위를 넓힙니다
라벨 오류 · 중복	발견 즉시 제거 목록을 만들고 전처리에 고정합니다

전처리와 증강 전략

전처리

모든 실험 고정

어노테이션 형식 단일화

손상 이미지와 라벨 오류 제거

비율 유지 리사이즈와 패딩

EXIF 회전 보정 · 픽셀 정규화

전처리가 실험마다 달라지면 비교 자체가 성립하지 않습니다.

증강

실험 변수

좌우·상하 반전, 소각도 회전

기본 세트

밝기·대비·색조 변화

기본 세트

Mosaic · MixUp

추가 실험

블러 · 노이즈 · 반사 모사

오류 분석 후 판단

한 번에 하나씩 켜고 효과를 확인합니다. 동시에 여러 개를 바꾸면 원인을 알 수 없습니다.

SECTION 04

모델 개발

한 번에 하나씩 바꾸고, 바꾼 것을 기록합니다.

전체 파이프라인



↩ 반복 루프 — 오류 분석 결과가 다음 실험의 설정을 바꾸고, 재학습으로 되돌아갑니다.

베이스라인 설계

최고 점수를 내리는 실험이 아니라, 이후 모든 실험을 비교할 **기준선**을 만드는 실험입니다.

```
exp_000_baseline.yaml

model: yolo - 소형 스케일
weights: COCO 사전 학습
imgsz: 640
epochs: 50
batch: 16
optimizer: 기본값
augment: off
```

기본 설정을 그대로 씁니다

튜닝을 섞으면 무엇 덕분에 좋아졌는지 알 수 없습니다.

가장 작은 모델부터 시작합니다

학습이 빨라 파이프라인 오류를 일찍 발견할 수 있습니다.

첫 제출까지 한 번에 끝냅니다

학습부터 Kaggle 제출까지 경로 전체가 동작하는지 먼저 확인합니다.

모델 선택 트레이드오프

상대 비교 예시

모델 스케일	정확도	학습 시간	이 프로젝트에서의 역할
소형 (n / s)	낮음	짧음	아이디어를 빠르게 걸러 내는 탐색용
중형 (m)	중간	보통	주력 — 대부분의 실험을 여기서 진행
대형 (l / x)	높음	길다 — 실험 횟수 감소	최종 후보 설정에만 적용
Transformer 계열	경우에 따라 높음	길고 튜닝이 까다로움	시간이 남을 때 비교용 1회 (선택 확장)

시간이 정해진 프로젝트에서는 **모델 크기보다 실험 횟수**가 결과를 더 크게 좌우합니다.

파인튜닝 전략

STEP 1

헤드만 학습

앞쪽 특징 추출부는 고정하고 분류·박스 예측 부만 학습합니다. 사전학습 지식을 지키면서 우리 클래스에 맞춥니다.



STEP 2

전체 낮은 학습률 학습

고정을 풀고 전체를 작은 학습률로 함께 학습합니다. 큰 학습률은 사전학습 지식을 지웁니다.



STEP 3

스케줄러로 마무리

학습 후반에 학습률을 점차 줄여 수렴시킵니다. 검증 성능이 가장 좋은 체크포인트를 저장합니다.

과적합 신호

학습 손실은 계속 내려가는데 검증 mAP가 정체하거나 떨어지면 조기 종료를 검토합니다.

체크포인트 정책

마지막 epoch가 아니라 검증 성능 최고 시점을 기준으로 저장하고 비교합니다.

하이퍼파라미터 실험 매트릭스

영향이 크고 검증이 빠른 변수부터, 한 번에 하나씩 바꿉니다.

순서	변수	탐색 범위 (예시)	확인할 것
1	learning rate	1e-4 / 5e-4 / 1e-3	손실 곡선이 안정적으로 내려가는가
2	image size	512 / 640 / 800	작은 알약의 재현율이 오르는가
3	증강 조합	기본 / +Mosaic / +색상	촬영 조건이 다른 이미지에서 개선되는가
4	batch size · optimizer	8 / 16 / 32 · SGD, AdamW	수렴 속도와 메모리 한계
5	conf · NMS 임계값	conf 0.15-0.4 · NMS 0.5-0.7	재학습 없이 조정 — 별도 항목으로 기록

로컬 검증 설계와 지표 세트

분할 규칙

train 80% · validation 20%

클래스 비율을 유지한 계층 분할

고정 seed로 분할 재현

같은 분할이어야 실험끼리 비교됩니다

유사 이미지 누수 차단

같은 촬영 세션은 한쪽에만 배치합니다

분할 규칙을 도중에 바꾸면 이전 실험 결과를 모두 버려야 합니다.

매 실험 기록 지표

mAP@0.5

mAP@0.5:0.95

Precision

Recall

클래스별 AP

학습 · 검증 손실

학습 시간

최고 체크포인트 epoch

항목이 하나라도 빠지면 그 실험은 비교 대상에서 제외합니다. 기록은 자동으로 남기고, 사람이 손으로 옮겨 적지 않습니다.

SECTION 05

실험 관리 시스템

모델과 함께 제출하는 핵심 산출물입니다. 이후 화면은 최종 UI가 아니라 구조 설명용 개념 다이어그램입니다.

실험 관리가 필요한 이유

도구가 없을 때

점수는 올랐는데 무엇을 바꿔서 올랐는지 모릅니다

가장 좋았던 체크포인트를 다시 찾지 못합니다

팀원마다 기록 형식이 달라 비교가 안 됩니다

같은 실험을 두 사람이 중복해서 돌립니다

도구가 있을 때

모든 실험이 설정 파일과 함께 자동으로 남습니다

두 실험의 차이를 화면에서 바로 비교합니다

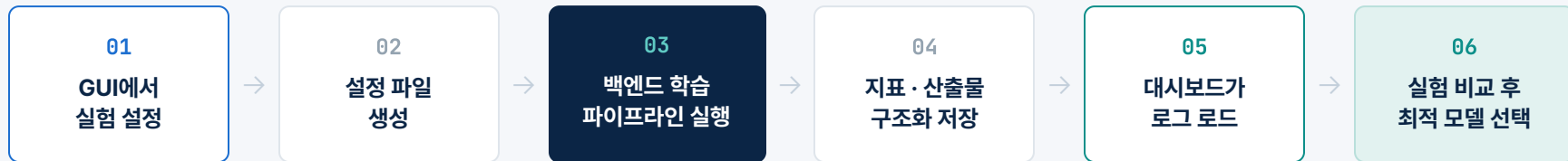
체크포인트와 지표, 제출 파일이 한 곳에 묶입니다

비전공 팀원도 실험을 등록하고 결과를 읽을 수 있습니다

이 도구는 편의 기능이 아니라, **실험 결과를 신뢰할 수 있게 만드는 최소 장치**입니다.

시스템 구조

화면은 **설정을 만들고 결과를 읽는** 역할만 합니다. 학습 실행은 백엔드 파이프라인이 담당합니다.



프론트엔드가 하는 일

설정 생성 · 실험 등록 · 로그 조회 · 비교 · 제출 파일 내려받기

프론트엔드가 하지 않는 일

모델 학습 로직 · 추론 연산 · 데이터 저장소 관리

GUI 화면 ① 실험 설정

개념 목업

새 실험 만들기 exp_014

모델

아키텍처

YOLO

모델 크기

m

사전학습 가중치

COCO

학습 파라미터 — 재학습 필요

learning rate

5e-4

batch · epochs

16 · 80

image size

800

optimizer ·
schedulerAdamW ·
cosine

증강 · 추론 임계값

flip ✓

HSV ✓

mosaic ✓

mixup

confidence

0.25

NMS IoU

0.60

점선 항목은 재학습 없이 변경 가능

설정 저장

실험 등록

GUI 화면 ② 실험 목록과 비교

예시 수치 · 개념 목업

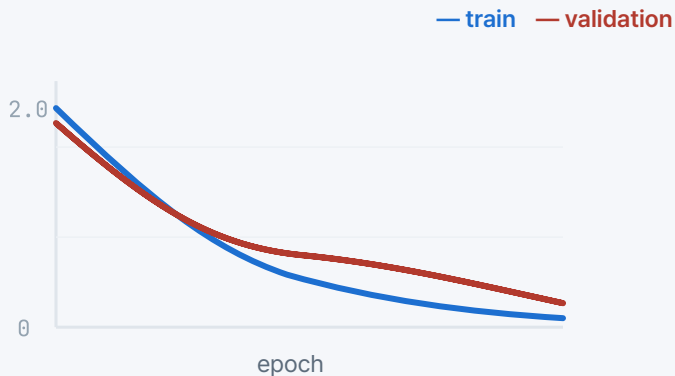
실험 ID	변경한 항목	mAP@0.5	mAP@.5:.95	Recall	학습 시간	Kaggle
exp_000	베이스라인	0.712	0.451	0.688	0:42	0.694
exp_004	imgsz 640 → 800	0.769	0.503	0.742	1:18	0.751
exp_009	+ mosaic 증강	0.774	0.518	0.781	1:26	0.768
exp_014	lr 1e-3 → 5e-4	0.803	0.547	0.796	1:24	0.789
exp_017	+ mixup 증강	0.781	0.522	0.803	1:31	미제출

“변경한 항목” 열이 이 화면의 핵심입니다. 무엇을 바꿨는지 없이 숫자만 남으면 실험 기록의 의미가 사라집니다.

대시보드 ① 학습 곡선

예시 수치

Loss



검증 손실이 학습 손실과 벌어지기 시작하면 과적합을 의심합니다.

mAP@0.5



두 실험을 겹쳐 그려, 최고 성능 시점과 그 체크포인트를 함께 고릅니다.

learning rate

스케줄러 적용 이력

Precision · Recall

epoch별 추이

학습 시간 · GPU

자원 사용 기록

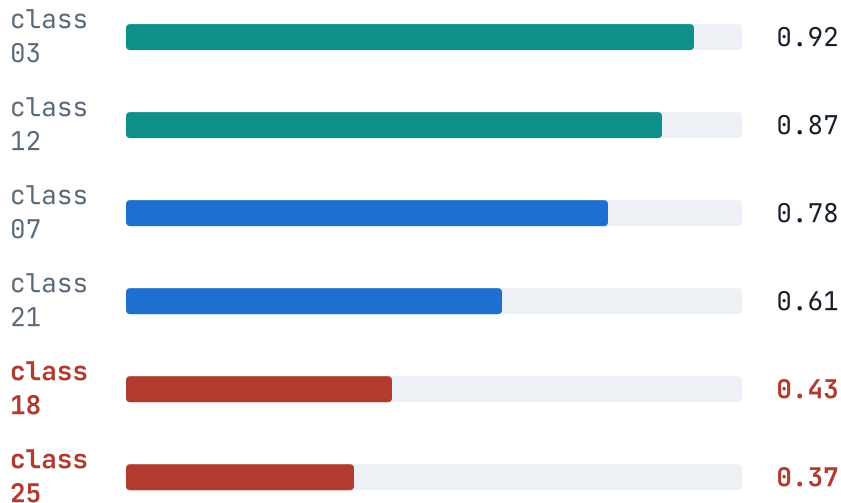
best checkpoint

epoch · 파일 경로

대시보드 ② 클래스별 성능과 혼동행렬

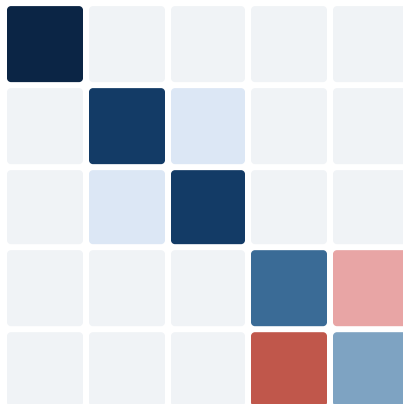
예시 수치

클래스별 AP@0.5



전체 mAP 하나로는 어떤 알약이 문제인지 알 수 없습니다.

혼동행렬



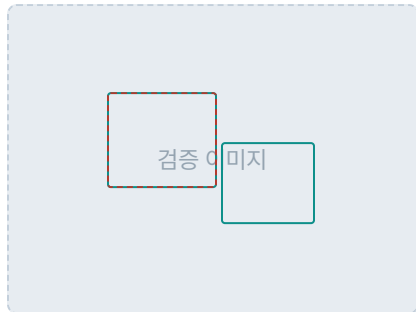
예측 →

↓ 정답

class 18과 25가 서로 혼동됩니다. 색과 크기가 비슷한 알약인지 실제 이미지로 확인합니다.

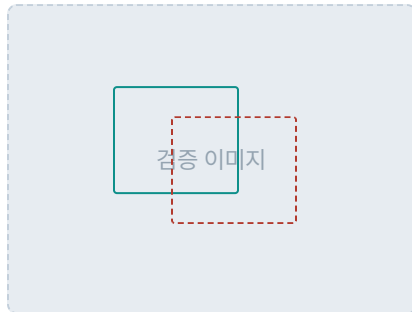
대시보드 ③ 실패 케이스 갤러리

— 정답 — 예측



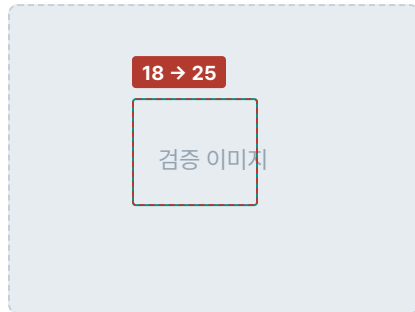
미탐 · 작은 객체

두 번째 알약을 아예 찾지 못했습니다



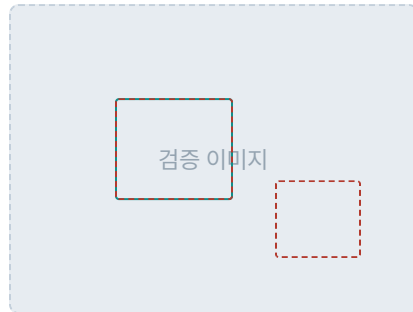
위치 오차 · 겹침

겹친 알약을 하나로 묶어 예측했습니다



클래스 혼동

위치는 맞았지만 종류를 틀렸습니다



오탐 · 반사

빛 반사를 알약으로 잘못 잡았습니다

실패 유형별 개수를 세면 다음에 무엇을 바꿔야 하는지가 정해집니다. 미탐이 많으면 해상도와 임계값을, 오탐이 많으면 증강과 NMS를 봅니다.

TensorBoard 로그와 커스텀 대시보드

대체하는 것이 아니라 위에 있는 구조입니다. TensorBoard 로그는 그대로 남깁니다.

TensorBoard 로그

표준 · 학습 중 자동 기록

epoch별 손실, 지표, 학습률을 실시간으로 남깁니다. 학습이 제대로 돌고 있는지 확인하는 용도입니다.

한계 — 실험 간 맥락(무엇을 왜 바꿨는지)과 실패 이미지가 남지 않습니다.

커스텀 대시보드

우리 프로젝트 전용 질문에 답하는 층

저장된 로그를 읽어 실험을 비교하고, 클래스별 성능과 실패 이미지, Kaggle 점수를 한 화면에 묶습니다.

답하려는 질문 — 어떤 실험이 최고인가 · 무엇을 바꿨나 · 어떤 클래스가 어려운가 · 로컬과 리더보드가 일치하는가

SECTION 06

실행과 마무리

무엇을 산출물로 내고, 최종 발표에서 무엇을 보여 줄 것인가.

오류 분석 루프

실험이 끝날 때마다 도는 고정 절차입니다. 가설을 **문장으로 적고** 하나만 바꿉니다.

01

지표에서 약점 찾기

클래스별 AP가 낮은 곳,
Precision과 Recall의 불균형



02

실패 이미지 유형 세기

미탐 · 오탐 · 위치 오차 · 클래스
혼동으로 분류



03

가설 한 문장 쓰기

“작은 알약 미탐이 많으니 해상
도를 올리면 재현율이 오를 것”



04

한 가지만 바꿔 재학습

가설과 변경 항목을 실험 기록에
함께 남깁니다

↩ 가설이 맞았는지 확인하고 다시 01로 돌아갑니다. 틀린 가설도 기록에 남깁니다.

베이스라인과 개선 모델 비교

예시 수치 · 보고 형식

지표	베이스라인 exp_000	개선 모델 exp_014	변화
mAP@0.5	0.712	0.803	+0.091
mAP@0.5:0.95	0.451	0.547	+0.096
Recall	0.688	0.796	+0.108
학습 시간	0:42	1:24	2배 증가

보고에서 중요한 것은 오른쪽 숫자가 아니라 **그 변화를 만든 실험이 무엇이었는지**입니다. 각 수치 옆에 해당 실험 ID를 함께 답니다.

Kaggle 제출 관리

제출 규칙

로컬 검증이 기준 이상 개선된 경우에만 제출합니다

모든 제출에 실험 ID와 체크포인트를 연결합니다

제출 파일은 대시보드에서 생성합니다 — 수작업 금지

로컬 점수와 리더보드 점수를 항상 짝지어 기록합니다

제출 기록 형식

```
submission_id sub_007
experiment exp_014
checkpoint best_epoch_62.pt
conf / nms 0.25 / 0.60
local mAP@.5 0.803
kaggle score 0.789
```

이 기록이 있어야 “리더보드 점수가 올랐다”를 “무엇이 올랐다”로 바꿔 말할 수 있습니다.

리더보드 과적합 경계

제출 점수만 보고 설정을 조금씩 고치면, 모델이 아니라 **시험지에 맞춰집니다.**

과적합 신호

로컬 검증은 그대로인데 리더보드만 조금씩 오릅니다

근거 없이 임계값만 바꿔 가며 여러 번 제출합니다

왜 좋아졌는지 설명하지 못하는 설정이 최고 점수입니다

우리 규칙

로컬 검증 점수를 항상 함께 보고합니다

로컬이 개선되지 않은 설정은 채택하지 않습니다

최종 모델은 리더보드 순위가 아니라 두 점수가 함께 좋은 것으로 고릅니다

MVP와 선택 확장

MVP · 필수 반드시 완성

YOLO 베이스라인 학습과 첫 Kaggle 제출

파인튜닝과 하이퍼파라미터 실험 (10회 이상)

실험 설정 GUI와 설정 파일 생성

실험 비교 · 학습 곡선 · 클래스별 성능 대시보드

실패 케이스 갤러리와 제출 파일 생성

선택 확장 시간이 남으면

GUI에서 직접 학습 실행 트리거

실시간 학습 진행 모니터링

자동 하이퍼파라미터 탐색

Transformer 계열 모델 비교 실험

앙상블과 TTA 적용

최종 산출물

01

학습된 모델과 체크포인트

최종 선정 모델 파일과, 선정 근거가 되는 검증 지표

02

전체 실험 기록

모든 실험의 설정 파일, 지표, 로그, Kaggle 점수

03

실험 관리 웹 애플리케이션

실험 설정 GUI와 결과 대시보드. 모델과 함께 핵심 산출물입니다.

04

재현 가능한 학습 코드

설정 파일 하나로 같은 결과를 다시 만들 수 있는 파이프라인

리스크와 한계

데이터 불균형

이미지가 적은 클래스는 성능 상한이 낮습니다.
증강으로 일부만 완화됩니다.

GPU 시간 제약

돌릴 수 있는 실험 횟수가 제한됩니다. 큰 모델을
쓸수록 더 줄어듭니다.

라벨 품질

어노테이션 오류가 있으면 모델이 아니라 데이터
가 성능 한계를 만듭니다.

GUI 범위 확대

기능을 늘리다 모델 실험 시간이 줄어들 수 있습
니다. MVP 목록을 기준으로 자릅니다.

검증과 실제의 차이

로컬 검증 분포가 대회 평가 데이터와 다를 수 있
습니다.

의료적 사용 불가

이 모델은 실험 목적입니다. 실제 복약 판단에 사
용할 수 있는 수준의 검증을 거치지 않았습니다.

최종 데모 시나리오

최종 발표에서 화면으로 보여 줄 순서 — 약 5분

01 YOLO 모델과 사전학습 체크포인트 선택

02 학습률 · 배치 · 이미지 크기 · epoch · 증강 설정

03 실험 등록 — 설정 파일 생성 확인

04 완료된 실험의 대시보드 열기

05 베이스라인과 나란히 비교

06 Precision · Recall · mAP · 손실 곡선 확인

07 어려운 검증 이미지와 실패 유형 살펴보기

08 최적 체크포인트 선택

09 샘플 알약 이미지로 추론 실행

10 Kaggle 제출 파일 생성

정리

점수를 만드는 것이 아니라, 점수의 근거를 만듭니다

한 번에 하나만

두 개를 동시에 바꾸면 원인을 알 수 없습니다

바꾼 것을 기록

기록되지 않은 실험은 없었던 실험입니다

실패 이미지를 직접

숫자만 보면 다음에 무엇을 할지 알 수 없습니다